

АБСТРАКНИ КЛАСОВЕ – Задача Instruments

```
Microsoft Visual Studio Debug Console

{
    Музикални инструменти
}

Пиано свири нота До.
Тип: Клавишен

Орган свири нота До.
Тип: Клавишен

Китара свири нота До.
Тип: Струнен

Бас китара свири нота До.
Тип: Струнен

{
    Музикални инструменти
}

Пиано свири нота Ре.
Тип: Клавишен

Китара свири нота Ре.
Тип: Струнен

D:\Exercises\11 class\Inheritance\Abstract_
ith code 0.
Press any key to close this window . . .
```

Условието е от г-жа В. Маринова, като ще разгледаме няколко примерни решения.

1. Създайте абстрактен клас **Instrument**, който има:
 - свойство `string Name`
 - абстрактен метод `void PlayNote(string note)`
 - абстрактен метод `string GetTypeNames()`
2. Създайте два наследника:
 - Piano
 - Guitar
3. Наследниците трябва:
 - да използват свой собствен начин за изпълнение на ноти (например: „Пианото свири нота До.“)
 - да върнат тип на инструмента (напр. „Клавишен“ или „Струнен“)
4. В `Main()`:
 - създайте **масив** от 3–4 инструмента (смесени видове)
 - за всеки:
 - извикайте `PlayNote("Do")`
 - отпечатайте типа, връщан от `GetTypeNames()`

Нека създадем абстрактен родителски клас с 2 конструктора в него – разен и друг, който обработва име:

```
abstract class Instrument
{
    4 references
    public string Name { get; set; }
    4 references
    public abstract void PlayNote(string note);
    4 references
    public abstract string GetTypeNames();
    конструктори
    0 references
    1) protected Instrument() може да създава празни обекти
    {
    }
    1 reference изисква име
    2) protected Instrument(string name)
    {
        Name = name;
    }
}
```

Нека създадем наследник `Piano`, който ще наследи единия конструктор и за по-интересно, нека един от методите в него `GetTypeNames` връща текст, вместо да го отпечата като `DisplayInfo`, за да упражним и този начин:

```

class Piano : Instrument
{
    3 references
    public Piano(string name) : base(name) // само клас Piano наследява конструктор
    {
    }
    3 references
    public override void PlayNote(string note)
    {
        Console.WriteLine($"{Name} свири нота {note}.");
    }

    3 references
    public override string GetTypeName() => "Клавишен"; // съкратено вместо { return }
    // а не void -> CW
}

```

В другия наследник умишлено не слагаме ctor, за да потренираме и създаване на обекти без него:

```

3 references
class Guitar : Instrument
{
    3 references
    public override void PlayNote(string note)
    {
        Console.WriteLine($"{Name} свири нота {note}.");
    }

    3 references
    public override string GetTypeName() => "Струнен";
}

```

Остава да създадем масив в Main() – при масив винаги размерът трябва да бъде предварително известен и конкретен, или директно посяване на нови обекти още при създаването му, както е в случая. Ако се наложи редакция, може по-късно с `resize`. кодът е достатъчно описан с коментари:

```

static void Main()
{
    4
    // масив
    Instrument[] instruments = {
        new Piano ("Пиано"), // с конструктор
        new Piano ("Орган"),
        new Guitar { Name = "Китара" }, // без конструктор
        new Guitar { Name = "Бас китара" }
    };

    foreach (var i in instruments) // вместо var може Instrument – сложен съставен тип
    {
        i.PlayNote("До");
        Console.WriteLine($"Tun: {i.GetTypeName()}");
        Console.WriteLine(new string('~', 39));
    }
}

```

Ако искаме да упражним списък, ще видим колко по-лесен и гъвкав е – създава се с нов родителски клас и по всяко време, винаги можем да подадем нови обекти на наследниците към него с `Add`, колкото искаме на брой

```
List<Instrument> instrumentsList = new List<Instrument>();
instrumentsList.Add(new Piano("Пиано")); // с конструктор
instrumentsList.Add(new Guitar { Name = "Китара" }); // без конструктор

foreach (var i in instrumentsList)
{
    i.PlayNote("Pe");
    Console.WriteLine($"Tun: {i.GetTypeName()}");
    Console.WriteLine(new string('~', 39));
}
```

Обхождането с foreach е еднакво. Могат да се отделят методи извън Main(), но понеже вече има разделители между самите обекти, аз реших да си поиграя със заглавие:

```
private static void DecoratedTitle()
{
    Console.WriteLine("
Музикални инструменти
");
}
```

Сравнение между масиви и списъци:

Характеристика	Масив (Array)	Списък (List<T>)
Размер	Фиксиран	Динамичен
Добавяне	Не поддържа .Add()	 Поддържа .Add()
Преоразмеряване	Ръчно (Array.Resize)	Автоматично
Производителност	По-бърз достъп	Малко по-бавна
Удобство	По-сложно	Много лесно
Предимства	По-малко памет	Гъвкавост

Целият код:

```
class Program
{
    static void Main()
    {
        DecoratedTitle();

        Instrument[] instruments = {
            new Piano ("Пiano"),           // с конструктором
        };
    }
}
```

```

        new Piano ("Орган"),
        new Guitar { Name = "Китара" },    // без конструктор
        new Guitar { Name = "Бас китара" }
    };

    foreach (var i in instruments) // вместо var може Instrument - сложен
съставен тип
    {
        i.PlayNote("До");
        Console.WriteLine($"Тип: {i.GetTypeName()}");
        Console.WriteLine(new string('~', 39));
    }

    DecoratedTitle();

    List<Instrument> instrumentsList = new List<Instrument>();
    instrumentsList.Add(new Piano("Пиано")); // с конструктор
    instrumentsList.Add(new Guitar { Name = "Китара" }); // без конструктор

    foreach (var i in instrumentsList)
    {
        i.PlayNote("Ре");
        Console.WriteLine($"Тип: {i.GetTypeName()}");
        Console.WriteLine(new string('~', 39));
    }
}

private static void DecoratedTitle()
{
    Console.WriteLine("┌────────────────────────────────────────────────────────────────────────────────┐");
    Console.WriteLine("│                                Музикални инструменти                                │");
    Console.WriteLine("└────────────────────────────────────────────────────────────────────────────────┘");
}

}

abstract class Instrument
{
    public string Name { get; set; }
    public abstract void PlayNote(string note);
    public abstract string GetTypeName();

    protected Instrument()
    {
    }
    protected Instrument(string name)
    {
        Name = name;
    }
}

class Piano : Instrument
{
    public Piano(string name) : base(name) // само клас Piano наследява
конструктор
    {
    }
    public override void PlayNote(string note)
    {
        Console.WriteLine($"{Name} свири нота {note}.");
    }
}

```

```
    }

    public override string GetTypeName() => "Клавишен"; // съкратено вместо {
return }
}

class Guitar : Instrument
{
    public override void PlayNote(string note)
    {
        Console.WriteLine($"{Name} свири нота {note}.");
    }

    public override string GetTypeName() => "Струнен";
}
```